

Contents

Module 7 — UserForms and Controls in VBA	1
Learning outcomes	1
100-minute sequence	1
1. When is a form useful?	1
2. Create the UserForm	2
3. Property, method, and event	2
4. Display the form	2
5. Calculate button event	3
6. Clear, Close, and Initialize events	3
7. Practice and testing	4
8. Interface design boundaries	4
Short exercise — 3 questions	4
Final checklist	5
Closing note	5
Further reading	5

Module 7 — UserForms and Controls in VBA

Learning outcomes

Students will be able to:

- explain the role of a UserForm as an interface;
- add and name controls;
- distinguish properties, methods, and events;
- write a button event handler;
- validate form input; and
- reuse a Function from a standard module.

100-minute sequence

Minutes	Activity
0–10	Compare worksheet input with form input
10–25	Create a UserForm and add controls
25–40	Name, Caption, Value, properties, and events
40–58	Demonstrate the Calculate button event
58–76	Build a concrete-volume form
76–86	Validation, Clear button, and Close button
86–97	Three-question short exercise and peer demonstration
97–100	Save, compile, and complete final checklist

Minimum product: one UserForm with three inputs, one output, and Calculate, Clear, and Close buttons; invalid input must produce a clear message.

1. When is a form useful?

Direct worksheet input is appropriate for large tables and data that should remain visible together. A UserForm is useful when it should:

- guide the user through a small set of inputs;

- constrain order and choices;
- reduce the chance of editing formulas; and
- provide validation messages before storing data.

A form does not automatically make a program correct. Formulas, units, validation, and testing remain essential.

2. Create the UserForm

1. Open the Visual Basic Editor with Alt+F11.
2. Choose Insert → UserForm.
3. In the Properties Window, change (Name) to frmVolume.
4. Change Caption to Concrete Volume Calculator.
5. Add controls from the Toolbox.

Required controls

Type	(Name)	Caption/purpose
Label	lblLength	Length (m)
TextBox	txtLength	length input
Label	lblWidth	Width (m)
TextBox	txtWidth	width input
Label	lblHeight	Height (m)
TextBox	txtHeight	height input
Label	lblResult	Volume: —
CommandButton	cmdCalculate	Calculate
CommandButton	cmdClear	Clear
CommandButton	cmdClose	Close

Names such as txtLength communicate both control type and meaning, unlike a default name such as TextBox1.

3. Property, method, and event

- A **property** describes state or an attribute: Caption, Name, Value, Enabled.
- A **method** requests an action: SetFocus, Show, Hide.
- An **event** is something the program responds to: Click, Change, Initialize.

```

user clicks button
    ↓
cmdCalculate_Click event
    ↓
read TextBox Value properties
    ↓
call RectangularVolume Function
    ↓
change lblResult Caption property
  
```

4. Display the form

In a standard module:

Option Explicit

```
Public Sub OpenVolumeForm()  
    frmVolume.Show  
End Sub
```

Run OpenVolumeForm, or assign it to a worksheet button.

5. Calculate button event

Double-click cmdCalculate and add:

```
Private Sub cmdCalculate_Click()  
    Dim length_m As Double  
    Dim width_m As Double  
    Dim height_m As Double  
    Dim volume_m3 As Double  
  
    If Not IsNumeric(Me.txtLength.Value) Or _  
        Not IsNumeric(Me.txtWidth.Value) Or _  
        Not IsNumeric(Me.txtHeight.Value) Then  
        MsgBox "All dimensions must be numeric.", vbExclamation  
        Exit Sub  
    End If  
  
    length_m = Cdbl(Me.txtLength.Value)  
    width_m = Cdbl(Me.txtWidth.Value)  
    height_m = Cdbl(Me.txtHeight.Value)  
  
    volume_m3 = RectangularVolume(length_m, width_m, height_m)  
  
    If volume_m3 < 0 Then  
        MsgBox "All dimensions must be greater than zero.", vbExclamation  
        Exit Sub  
    End If  
  
    Me.lblResult.Caption = "Volume: " & _  
        Format(volume_m3, "0.000") & " m3"  
End Sub
```

Me refers to the UserForm containing this code. RectangularVolume reuses the public function created in Module 6.

6. Clear, Close, and Initialize events

```
Private Sub cmdClear_Click()  
    Me.txtLength.Value = ""  
    Me.txtWidth.Value = ""  
    Me.txtHeight.Value = ""  
    Me.lblResult.Caption = "Volume: —"  
    Me.txtLength.SetFocus  
End Sub  
  
Private Sub cmdClose_Click()
```

```

        Unload Me
End Sub

Private Sub UserForm_Initialize()
    Me.lblResult.Caption = "Volume: -"
    Me.txtLength.Value = ""
    Me.txtWidth.Value = ""
    Me.txtHeight.Value = ""
End Sub

```

7. Practice and testing

Case	L	W	H	Expected result
Normal	10	2	0.3	6.000 m ³
Zero	0	2	0.3	dimensions must be > 0
Text	ten	2	0.3	input must be numeric
Fraction	1.5	0.2	0.4	0.120 m ³

One-minute individual demonstration:

1. open the form;
2. run the normal case;
3. enter one invalid input;
4. explain the event that runs; and
5. show the reused Function.

8. Interface design boundaries

- always show units;
- use a logical tab order;
- do not rely on colour as the only message;
- tell the user what is wrong and how to fix it;
- do not store results before input is valid; and
- provide a way to cancel or close the form.

Short exercise — 3 questions

1. Predict

The user enters abc in txtLength and clicks Calculate. Put the important statements in execution order and explain why CDb1 is not called.

2. Fix

The Clear button empties all three TextBoxes, but the old result remains visible. Add the required statement and name the property being changed.

3. Explain

Classify these as property, method, or event: Caption, SetFocus, Click, Value, Show, Initialize. Explain how the three categories interact when Calculate is clicked.

1. `cmdCalculate_Click` starts; `IsNumeric` returns `False`; a message appears; `Exit Sub` ends the procedure. `Cdbl` is never reached, avoiding a type mismatch.
2. `Me.lblResult.Caption = "Volume: -"`; the Label's `Caption` property changes.
3. Properties: `Caption`, `Value`; methods: `SetFocus`, `Show`; events: `Click`, `Initialize`. An event triggers a handler, which reads or changes properties and may call methods.

Final checklist

- ☐ Every control has a meaningful name.
- ☐ I distinguish properties, methods, and events.
- ☐ Validation occurs before `Cdbl` and before calculation.
- ☐ The form reuses a Function from a standard module.
- ☐ I can demonstrate both a normal and an invalid case.

Closing note

A UserForm connects the user to an algorithm. A good interface guides input, but result quality still depends on a correct function, clear validation, and evidence from testing.

Further reading

1. Michael Alexander & Dick Kusleika, *Excel 2019 Power Programming with VBA*, Wiley, 2019.
2. Bill Jelen & Tracy Syrstad, *Microsoft Excel VBA and Macros (Office 2021 and Microsoft 365)*, Microsoft Press, 2022.
3. Steve Krug, *Don't Make Me Think, Revisited: A Common Sense Approach to Web Usability*, 3rd ed., New Riders, 2014.
4. Ben Shneiderman et al., *Designing the User Interface: Strategies for Effective Human-Computer Interaction*, 6th ed., Pearson, 2016.

← Module 6 · Module list